

MAS FINAL PROJECT: OVERVIEW AND ARCHITECTURE

1. PROJECT PURPOSE

This project implements a deterministic multi-agent warehouse system where robots

- ? navigate on a 2D grid,
- ? pick requested shelves,
- ? deliver requested shelves to goal cells,
- ? avoid collisions through cooperative planning,
- ? improve safety through verification-guided refinement.

The system is intentionally algorithmic (not ML-policy-driven).

Core claim: practical multi-agent coordination can be built with explicit planning, symmetry-reduced verification + constraint refinement.

2. WHAT THE SYSTEM CONTAINS

Core runtime modules:

- ? main.py: CLI entrypoint and orchestration.
- ? env.py: environment dynamics, reward/collision handling, rendering, evaluation.
- ? agent.py: robot/action primitives and pick/drop semantics.
- ? pathfinding.py: cooperative planner and assignment logic.
- ? symmetry_reduction.py: role-orbit detection and canonical state mapping.
- ? verification.py: bounded safety verification over quotient states.
- ? refinement.py: conversion of unsafe traces into planner constraints.
- ? config.py + config.yaml: defaults and runtime overrides.

Support:

- ? tests/: module and integration tests.
- ? docs/: documentation site and written docs.

3. ARCHITECTURAL STYLE

The architecture is a deterministic closed loop:

1. Planner computes one action per robot.
2. Environment executes actions simultaneously with conflict resolution.
3. Verification optionally runs bounded trials on symmetry-reduced state keys.
4. Refinement optionally injects hard constraints back into planner.

This forms a CEGAR-like pattern:

- ? generate behavior,
- ? detect violations,
- ? refine constraints,
- ? repeat.

4. EXECUTION MODES (FROM MAIN.PY)

The CLI supports:

- ? interactive: continuous render + planning loop.
- ? simulate: episode loop with optional rendering.
- ? eval: collision-rate evaluation over episodes.
- ? --detect-symmetry: print detected orbits and quotient metadata.
- ? --verify-refine: run verification and apply refinement iteratively.

Main orchestration functions:

- ? run_interactive(...)
- ? run_simulation(...)
- ? run_eval(...)
- ? run_verify_refine(...)

Default resolution:

- ? _apply_defaults(...) binds CLI omissions to values from config.py.

5. ENVIRONMENT MODEL (ENV.PY)

5.1 STATE VARIABLES

WarehouseEnv manages:

- ? grid dimensions (grid_w, grid_h),
- ? goal cells (GOALS),
- ? robots (self.robots),
- ? shelves (self.shelves),
- ? step counters and statistics,
- ? planner debug payloads:
 - _planner_allowed_shelf_entries,
 - _planner_last_actions,
 - _planner_debug_by_agent,
- ? conflict/event telemetry:
 - last_conflicts,
 - event_log,
 - traffic_heat,
 - collision_heat.

5.2 RESET AND SPAWN

reset():

- ? places robots randomly on free cells (excluding goals),
- ? places shelves randomly on free cells,
- ? initializes metrics/log buffers,
- ? returns per-robot normalized state vectors.

5.3 STEP SEMANTICS

step(actions, record_trajectories=False) performs one simultaneous timestep:

1. Decode integer actions into Action enum.
2. Record intent-level conflicts (boundary, vertex, edge) for diagnostics.
3. Apply non-forward actions (TURN_LEFT, TURN_RIGHT, PICK_DROP, WAIT).
4. Collect forward intents.
5. Reject forward moves that violate:
 - boundary,
 - shelf occupancy rules,
 - static occupied targets,
 - same-target contention,
 - edge swap conflicts.
6. Apply valid forward moves simultaneously.
7. Compute shaping rewards from distance-to-target delta.
8. Apply delivery team bonus and penalties.
9. Update episode/global counters and logs.

done is currently bounded by `self.steps > 1000`.

5.4 RENDERING AND OBSERVABILITY

`render()` draws:

- ? grid, goals, shelves, robots,
- ? orbit-color indicator using symmetry detection,
- ? conflict overlays.

Rendering is presentation-only and does not alter simulation logic.

5.5 EVALUATION

`evaluate(planner, num_episodes, max_steps_per_episode, ...)` runs no-render episode returns:

- ? average collisions per agent per episode.

6. AGENT SEMANTICS (AGENT.PY)

6.1 ENUMS

- ? Direction: UP, RIGHT, DOWN, LEFT.
- ? Action: FORWARD, TURN_LEFT, TURN_RIGHT, PICK_DROP, WAIT.

6.2 ROBOT STATE

Each Robot has:

- ? id, x, y, dir,
- ? optional carrying shelf dict.

6.3 MOTION AND INTERACTION RULES

- ? `forward(env)`: moves one cell if inside bounds and not blocked by robot/shelf

- ? pick_or_drop(env):
- picks shelf on same cell if empty-handed,
- delivers requested shelf at goal,
- otherwise drops carried shelf.

Shelf-blocking policy is planner-aware:

- ? _occupied_by_shelf(...) allows entry to assigned requested shelf cell only,
- ? reads allowed entries from env._planner_allowed_shelf_entries.

This is a key planner-environment contract.

7. COOPERATIVE PLANNING (PATHFINDING.PY)

CooperativePlanner is the central policy module.

7.1 INTERNAL COMPONENTS

- ? ReservationTable: time-indexed vertex and edge occupancy reservations.
- ? ConstraintTable: hard forbidden vertex/edge constraints.
- ? AssignmentManager: robot-to-shelf and shelf-to-robot matching.
- ? IdleTracker: starvation/deadlock mitigation through idle monitoring.

7.2 PLANNING STRATEGY

Per timestep:

1. Track idleness and update assignments.
2. Build planning order with rotating priority offset.
3. Plan robots sequentially in that order.
4. For each robot:
 - decide immediate PICK_DROP if applicable,
 - else run time-aware A* (astar_time) to target,
 - fallback to deterministic local escape action if stuck.
5. Reserve trajectory prefixes for already-planned robots.
6. Sanitize first-step conflicts (same target/swap/invalid forward).

7.3 A* SEARCH SPACE

Space-time node key:

- ? (x, y, dir, t).

Transition constraints include:

- ? grid bounds,
- ? blocked shelf cells,
- ? reservations (vertex + both edge directions),
- ? dynamic blocked occupancy assumptions,
- ? refinement constraints.

7.4 ASSIGNMENT LOGIC

`minimum_cost_matching(...)` (Hungarian algorithm) is used for:

- ? free robot -> requested shelf assignment,
- ? carrier -> delivery goal slot assignment.

This reduces assignment churn and keeps delivery flow balanced.

8. SYMMETRY REDUCTION (SYMMETRY_REDUCTION.PY)

Symmetry reduction maps many labeled states into one quotient representative.

8.1 ROLE-ORBIT DETECTION

Robots are grouped by compact role:

- ? (0,0): carrying nothing,
- ? (1,1): carrying requested shelf,
- ? (1,0): carrying non-requested shelf.

Robots with same role belong to same orbit.

8.2 CANONICALIZATION

For each orbit:

- ? encode each member as (x, y, dir, carrying_requested),
- ? sort tuples lexicographically,
- ? store orbit tuple.

State key is tuple of orbit tuples (plus optional shelf key).

Optional shelf inclusion:

- ? include uncarried shelf tuples (x, y, requested_flag) sorted.

8.3 QUOTIENT METADATA

`build_quotient_model(env)` provides:

- ? orbits,
- ? representative index per orbit,
- ? agent index -> orbit index mapping.

9. VERIFICATION (VERIFICATION.PY)

`verify_on_quotient(...)` performs bounded trial-based safety checking.

9.1 TRIAL LOOP

For each trial:

1. Reset environment.
2. Maintain visited set of canonical quotient keys.
3. At each step:
 - canonicalize current state,
 - stop trial if key repeated (cycle in quotient space),
 - execute planner action,
 - detect collisions/conflicts,
 - evaluate minimum pairwise Manhattan separation.

9.2 SAFETY OUTPUTS

Returns dict with fields such as:

```
? safe (bool),
? delta_q (minimum margin relative to required separation),
? avg_collision_rate,
? when unsafe:
- conflicts,
- counterexample,
- actions.
```

delta_q interpretation:

```
? delta_q >= 0: maintained separation requirement,
? delta_q < 0: violated required separation.
```

10. REFINEMENT (REFINEMENT.PY)

refine_planner_with_conflicts(...) converts verification conflicts into hard planner constraints.

Mapping:

```
? vertex conflict -> forbid that position at t+1,
? edge conflict -> forbid both edge directions at t+1,
? boundary conflict -> forbid out-of-bounds target position at t+1,
? separation currently ignored.
```

Fallback path:

```
? if no explicit conflicts but trace exists, add coarse position constraint for
- first same-cell collision if present,
- else closest pair anchor at time 1.
```

Result:

```
? returns count of applied constraints.
```

11. CONFIGURATION SYSTEM (CONFIG.PY + CONFIG.YAML)

config.py contains typed defaults for:

- ? grid size and goals,
- ? planner horizon and node caps,
- ? idleness and reservation windows,
- ? rendering FPS and visuals,
- ? verification thresholds and trial sizes,
- ? refinement iteration/constraint limits.

`load_from_yaml(...)` optionally overrides these values at startup.

`main.py` then applies CLI overrides on top.

Precedence is effectively:

1. hardcoded defaults,
2. YAML overrides,
3. CLI overrides.

12. DATA CONTRACTS BETWEEN MODULES

Critical contracts:

- ? Planner expects env fields:
 - robots, shelves, GOALS, grid size.
- ? Environment expects planner outputs:
 - list of integer action IDs aligned with `env.robots`.
- ? Agent shelf blocking depends on planner-provided:
 - `env._planner_allowed_shelf_entries`.
- ? Verification expects planner API:
 - `compute_actions(env)`.
- ? Refinement expects planner API:
 - `add_constraint_position(...)`,
 - `add_constraint_edge(...)`.

13. END-TO-END SAFETY LOOP

During `--verify-refine`:

1. Run `verify_on_quotient(...)`.
2. If `safe=True`, stop and report.
3. If `unsafe`:
 - extract conflicts/counterexample,
 - inject constraints via refinement,
 - rerun verification for next iteration.

This loop improves safety by hardening planner search space over iterations.

14. DETERMINISM AND REPRODUCIBILITY

- ? Seed control via `--seed` in `main.py`.
- ? Given same seed/config and deterministic code paths, execution is reproducible.
- ? Symmetry reduction is deterministic because canonicalization uses sorted tuples.

15. COMPLEXITY AND SCALABILITY NOTES

Major computational costs:

- ? Planner:
 - one bounded A* per robot per timestep,
 - plus assignment matching.
- ? Verification:
 - trials * horizon worst-case steps,
 - each step calls planner and environment dynamics.
- ? Symmetry:
 - canonicalization overhead is small relative to planning,
 - potential benefit depends on how frequently symmetric permutations occur.

16. FAILURE MODES AND LIMITS

Current design limitations:

- ? Bounded verification, not full proof of global safety.
- ? Symmetry reduction is role-based, not geometric-map automorphism-based.
- ? Refinement currently translates a limited conflict vocabulary.
- ? Performance sensitivity to congestion, horizon, and node caps.

These are engineering tradeoffs for tractability and transparency.

17. TESTING POSTURE

Existing tests include:

- ? unit/integration coverage across environment, agent, refinement, and verification
- ? integration case for verification+refinement (tests/test_verification_refinement.py)

Recommended additions:

- ? dedicated symmetry canonicalization unit tests,
- ? regression tests for quotient-key invariance under same-orbit permutations,
- ? performance baselines for quotient compression on selected seeded scenarios

18. ARCHITECTURE SUMMARY

The project is a layered deterministic MAS stack:

- ? execution substrate (env.py, agent.py),
- ? coordination policy (pathfinding.py),
- ? state-space reduction (symmetry_reduction.py),
- ? bounded safety analysis (verification.py),
- ? corrective feedback (refinement.py),
- ? orchestration/config (main.py, config.py).

The core value is not one isolated algorithm; it is the integrated pipeline that combines planning with formalized safety feedback under a symmetry-aware state abstraction.